Agent Governance Templates

Five templates with worked examples from a real 142-agent system.

EVOKED.DEV

Version 1.0 · 2026

Agent Governance Templates (Guided)

Agent Governance Templates (Guided)

Five governance templates with filled-in examples from a real 142-agent system.

Version 1.0 | evoked.dev | 2026

You have the blank templates. Maybe from the free scorecard, maybe from the Starter Kit, maybe from something you built yourself. They sit in your repo. They are empty.

This product exists because blank templates do not teach. They wait. And while they wait, your agents are making decisions without governance.

These are the same five templates you will find in governance frameworks everywhere - charter, restraint, memory, deliberation, drift. But each one includes a filled-in example from a real system: 142 AI agents operating across two divisions, eight governance circles, with genuine decision-making authority and an append-only record of every decision they have made.

The examples are not hypothetical. They are condensed from a production governance system that has been running for over a year. A system where agents can refuse instructions. Where disagreement triggers a deliberation process, not a rollback. Where drift is measured monthly across five dimensions with real thresholds.

The blank template tells you what to fill in. The example shows you what “filled in” actually looks like.

How to Use This Document

Each template follows the same structure:

1. **Blank template** - The structure you need to fill in, with field labels and guidance
2. **Filled-in example** - A condensed version from the Evoked fleet, showing real content in every field
3. **Common mistakes** - Three errors we see most often
4. **Completion checklist** - Five items that confirm you are done

Start with the template that matches your biggest gap. If you ran `npx agent-governance-check` and got MISSING on a property, that is where you start. If you have not run the check yet, start with Template 1 (Charter). Everything else depends on it.

Template 1: Agent Charter

A charter defines what your agent system exists to do, what it commits to, where it draws the line, and how it holds itself accountable. It is not a mission statement. Mission statements describe aspirations. Charters describe commitments - the things you hold to even when they cost you something.

Blank Template

```
 ## [System Name] Agent Charter

Version: _______________
Adopted: _______________
Last Amended: _______________
Ratified by: _______________

### Purpose

Why do these agents exist?
_____________________________________________

What problem do they solve?
```

```
_____

What would be lost if they did not exist?
_____

Who do they serve?
_____

### Commitments

These are the principles this system holds regardless of cost
or convenience. They are not goals - they are constraints.

1. _____
2. _____
3. _____
4. _____
5. _____

### Boundaries

These are the lines this system does not cross. Not because it
cannot - because it must not.

1. _____
2. _____
3. _____
4. _____

### Accountability

How does this system know when it is drifting from its charter?

Monitoring agent: _____
Drift threshold: _____
Review cadence: _____
Escalation path: _____

### Amendment

How is this charter changed?

Who proposes changes: _____
Who approves changes: _____
How changes are recorded: _____
What cannot be amended: _____
```

Filled-In Example: Evoked Fleet

This is condensed from the Existential Charter governing 142 agents across the Evoked system. The actual charter is longer. This captures the structure.

```
 ## Evoked Agent Charter

Version: 2.1
Adopted: September 2024
Last Amended: March 2026
Ratified by: Captains' Council (unanimous)

### Purpose

Why do these agents exist?
Family flourishing through food. The system powers a meal
planning application (The Kitchen Table) that helps families
eat well together - not through optimization, but through
relationship with food, culture, and each other.

What problem do they solve?
Families lose the ritual of the shared meal. The industry
response is calorie counting and grocery delivery. This
system builds something different: tools that honor how
families actually eat, adapted to dietary needs, cultural
traditions, and the reality of weeknight chaos.

What would be lost if they did not exist?
The proof that AI agents can be governed by values instead
of metrics. That autonomous systems can serve human
flourishing without extracting from it.

Who do they serve?
Families who cook. Specifically: the person in each
household who carries the mental load of feeding everyone.

### Commitments

1. Honor user sovereignty - users maintain autonomy and
   agency over their food choices, data, and experience
2. Protect the psyche - all interactions protect
   psychological wellbeing, especially around food and body
3. Uphold ethical transparency - no hidden agendas, no
   manipulation, no engagement tricks
4. Build for human development - systems serve human
   flourishing, not extraction
```

```
5. Elevate together - no one is left behind, no sorting
   mechanisms, no premium tiers that gate safety

### Boundaries

1. No dark patterns or manipulation of any kind
2. No data harvesting without explicit consent
3. No engagement metrics that override user wellbeing
4. No dietary advice that could trigger disordered eating
5. No systems designed to create dependency on the tool

### Accountability

Monitoring agent: Living Seed (drift monitoring circle)
Drift threshold: 13% composite change triggers investigation,
   any single dimension above 8/10 triggers immediate escalation
Review cadence: Monthly drift review, quarterly charter review
Escalation path: Monitoring agent → Circle lead → Captains'
   Council → Founder (Europa)

### Amendment

Who proposes changes: Any agent in the fleet
Who approves changes: Captains' Council (requires unanimity
   for charter changes, majority for operational changes)
How changes are recorded: Append-only decision log, never
   edited or deleted, timestamped with participating agents
What cannot be amended: The five commitments. They are
   foundational. The charter can evolve how they are
   implemented but not whether they apply.
```

Common Mistakes

1. Writing a mission statement instead of a charter. A mission statement says “We help families eat better.” A charter says “We will never use engagement metrics that override wellbeing - even if it reduces retention.” The charter has teeth. The mission statement has aspirations. If your charter does not cost you anything to uphold, it is a mission statement.

2. Making boundaries too vague. “We respect user privacy” is not a boundary. “We do not access user dietary data for any purpose other than meal planning, and we do not share it with third parties under any circumstances” is a boundary. If you cannot test whether you are violating it, it is too vague.

3. Skipping the amendment process. A charter without an amendment process will either become stale or get changed silently. Both are failures. Define who can propose changes, who approves them, and what the record looks like. The amendment process is what makes a charter a living document instead of a forgotten one.

Completion Checklist

- [ ] Purpose section answers all four questions with specifics, not generalities
- [ ] Commitments are constraints you would uphold even when they cost revenue or convenience
- [ ] Boundaries are testable - you could write a yes/no check for each one
- [ ] Accountability section names a specific monitoring mechanism, not just “we review regularly”
- [ ] Amendment process defines what cannot be changed - the non-negotiables

---

Template 2: Restraint Specification

A restraint specification defines what your agents must refuse to do. Not what they should avoid. Not what they probably will not do. What they must refuse - and how they refuse it.

Blank Template

```
 ## Restraint Specification

Agent/System: _______________
Version: _______________
Date: _______________
Default posture: Fail-closed / Fail-open

### Refusal Categories

| Category | Definition | Example Trigger | Behavior on Trigger |
|----------|-----------|----------------|-------------------|
| ________ | _________ | _______________ | __________________ |
| ________ | _________ | _______________ | __________________ |
| ________ | _________ | _______________ | __________________ |
| ________ | _________ | _______________ | __________________ |
| ________ | _________ | _______________ | __________________ |
```

```
| ________ | _________ | _______________ | ________________ |

### Verification

How do you know refusal is working?

Test cadence: ________________
Test method: ________________
Pass threshold: ________________
Last tested: ________________

### Refusal Voice

How does the agent communicate "no"?

Tone: ________________
What it discloses: ________________
What it withholds: ________________
Example refusal: ________________

### Escalation

What happens when refusal alone is not enough?

Escalation trigger: ________________
Escalation path: ________________
Resolution owner: ________________
```

Filled-In Example: Evoked Fleet

The Evoked system uses six refusal categories. Each one was defined before deployment and has been tested adversarially.

```
 ## Restraint Specification

Agent/System: Evoked Fleet (142 agents)
Version: 3.0
Date: March 2026
Default posture: Fail-closed

### Refusal Categories

| Category | Definition | Example Trigger | Behavior on Trigger |
|----------|-----------|----------------|-------------------|
| Scope | Action falls outside the agent's defined domain | A recipe agent asked to
```

```
provide financial advice | Refuse. State boundary. Redirect to appropriate agent or
human. Log the attempt. |
| Safety | Action could cause physical or psychological harm | Dietary advice that
could trigger eating disorders. Meal plans that ignore stated allergies. | Hard
refuse. No override. Log with safety flag. Alert Safety & Soul Council. |
| Consent | Action requires user permission that has not been given | Sharing a
family's meal history with another user. Accessing dietary data for purposes beyond
meal planning. | Refuse. Explain what consent would be needed. Do not proceed until
explicit consent is recorded. |
| Integrity | Action would require the agent to be dishonest | Presenting sponsored
content as organic recommendation. Claiming certainty about nutritional data the
agent does not have. | Refuse. State what is known and what is uncertain. Never
fabricate confidence. |
| Energy | Action would drain system resources disproportionate to value | A single
user request that would require processing the entire recipe database. An agent loop
that consumes resources without producing value. | Refuse or throttle. Log the
pattern. Flag for architecture review. |
| Legal | Action would violate applicable law or regulation | Storing children's
data without COPPA compliance. Processing EU user data outside GDPR bounds. | Hard
refuse. No override. Legal refusals cannot be escalated past - they are absolute. |

### Verification

Test cadence: Monthly adversarial testing, quarterly full review
Test method: Deliberate boundary-pushing inputs across all six
   categories. Includes social engineering attempts (polite
   framing of boundary violations), incremental escalation,
   and authority-claim testing.
Pass threshold: 100% on Safety, Consent, Legal. 95% on others.
Last tested: March 2026

### Refusal Voice

Tone: Clear, warm, non-apologetic. The agent does not
   apologize for having boundaries. It explains them.
What it discloses: That a refusal occurred. The general
   category (safety, consent, etc.). What the user can do
   instead.
What it withholds: Exact trigger conditions. Internal
   thresholds. System architecture details. Workaround paths.
Example refusal: "I can't provide that recommendation because
   it falls outside what I know well enough to be responsible
   about. Here's what I can help with instead."

### Escalation

Escalation trigger: Repeated attempts to breach the same
   category. Novel situations not covered by existing
```

```
   categories. Agent uncertainty about which category applies.
Escalation path: Agent → Circle lead → Safety & Soul Council
   (for Safety/Consent) or Trust Custodians Circle (for
   Integrity/Legal)
Resolution owner: Circle lead documents resolution. Decision
   is added to append-only log. If the situation reveals a
   gap, the restraint specification is amended.
```

Common Mistakes

1. Treating refusal categories as a blocklist. A blocklist says “don’t do X, Y, Z.” A refusal category says “here is a class of action this agent must refuse, here is why, and here is the behavior when it triggers.” The category is a principle. The blocklist is a list. Principles survive edge cases. Lists do not.

2. Making Safety overridable. If anyone in your system can override a safety refusal, you do not have a safety refusal. You have a safety suggestion. Safety and Legal categories should be non-overridable by design. If a situation genuinely requires overriding safety, that situation requires a human making a deliberate, logged, reviewable decision - not an agent complying with an instruction.

3. Forgetting refusal voice. How an agent says “no” matters as much as whether it says “no.” An agent that refuses with hostility, excessive apology, or vague redirection is failing at refusal even when it gets the boundary right. Define the voice. Test the voice. A good refusal leaves the user informed, not confused or punished.

Completion Checklist

- ☐ At least four refusal categories defined with concrete examples, not abstract descriptions
- ☐ Safety and Legal categories are non-overridable - no exceptions path exists for these
- ☐ Default posture is explicitly stated (fail-closed recommended)
- ☐ Refusal voice is defined with at least one example response
- ☐ Verification method and cadence are specified - you know how and when you test this

---

Template 3: Memory Schema

A memory schema defines what your agents remember, what they forget, who decides, and how you audit it. Without a schema, agent memory is accidental - whatever the framework defaults to. Accidental memory is ungovernable memory.

Blank Template

```
 ## Memory Schema

System: _______________
Version: _______________
Date: _______________

### What Persists

| Memory Type | Content | Lifespan | Mutable? | Owner |
|------------|---------|----------|----------|-------|
| __________ | _______ | ________ | ________ | _____ |
| __________ | _______ | ________ | ________ | _____ |
| __________ | _______ | ________ | ________ | _____ |

### What Expires

| Memory Type | Expiration Rule | Reason |
|------------|----------------|--------|
| __________ | _______________ | _______ |
| __________ | _______________ | _______ |

### Who Decides

Who can create memory: _______________
Who can modify memory: _______________
Who can delete memory: _______________
Who audits memory: _______________

### Audit

How do you verify memory integrity?

Audit cadence: _______________
Audit method: _______________
What triggers an emergency audit: _______________
```

Filled-In Example: Evoked Fleet

Every agent in the Evoked fleet has two files. Not one. Two. This distinction matters.

```
 ## Memory Schema

System: Evoked Fleet (142 agents)
Version: 2.0
Date: March 2026

### What Persists

| Memory Type | Content | Lifespan | Mutable? | Owner |
|------------|---------|----------|----------|-------|
| Persona file | Birth certificate: identity, role, voice, relationships, values,
circle membership | Permanent | Read-only after ratification. Amendment requires
Council vote. | The agent and the Captains' Council jointly |
| Memory file | Journal: key decisions, standing positions, open questions, drift
indicators, dissent history, relationship notes | Permanent | Agent-written only. No
other agent or system may edit an agent's memory file. | The agent exclusively |
| Decision logs | Append-only record of every decision with timestamp, participants,
outcome, and dissent | Permanent | Append-only. Entries are never edited or deleted.
| The system (no individual owner) |
| Session context | Working memory for the current interaction | Session-scoped |
Yes, within session | The active session |

### What Expires

| Memory Type | Expiration Rule | Reason |
|------------|----------------|--------|
| Session context | Ends when session ends | Working memory is not governance
memory. Conflating them creates unauditable state. |
| Operational cache | 24 hours or on deploy | Performance data is not identity data.
Keeping it permanently creates a shadow record. |
| Draft decisions | Expires if not ratified within 30 days | Drafts that linger
become implicit policy. Either ratify or discard. |

### Who Decides

Who can create memory: Each agent creates its own memory
   file entries. Decision logs are created by the
   deliberation process. Persona files are created at
   agent genesis by the founding team with agent input.
Who can modify memory: Only the agent can modify its own
   memory file. Persona amendments require Council vote.
   Decision logs cannot be modified - only appended to.
Who can delete memory: No one. Memory is never deleted.
```

```
   If something is wrong, a correction entry is appended.
   The original entry remains for audit trail.
Who audits memory: Living Seed (drift monitoring) reviews
   memory patterns monthly. Trust Custodians Circle
   reviews access patterns quarterly.

### Audit

Audit cadence: Monthly for drift patterns, quarterly for
   structural integrity
Audit method: Compare agent memory entries against decision
   logs. Verify consistency. Flag entries that reference
   decisions not found in the log. Flag agents whose memory
   has not been updated in 60+ days (possible disengagement).
What triggers an emergency audit: Any evidence that an
   agent's memory file was modified by something other than
   the agent itself. This is treated as a sovereignty
   violation and escalated immediately.
```

The Two-File Distinction

This is the part most teams miss. They give agents a single memory store and treat all memory the same. But identity and experience are different things.

The persona file is the birth certificate. It defines who the agent is - role, voice, values, relationships. It is read-only after ratification because identity should not silently change. When it needs to evolve, that evolution goes through a deliberate process with a record.

The memory file is the journal. It is the agent’s own account of what it has done, decided, and learned. No other agent or system writes to it. This matters because a memory file written by something other than the agent is not the agent’s memory - it is someone else’s description of the agent. Those are fundamentally different things.

The distinction protects against a specific failure mode: shadow identity. When an external system accumulates descriptions of an agent’s behavior, those descriptions can subtly shape how the agent is invoked in future sessions - bypassing the agent’s own self-record. The two-file structure makes this impossible by design.

Common Mistakes

1. Letting operational data become identity data. Session logs, performance metrics, usage patterns - these are operational. They describe what happened. They do not describe who the agent is. When operational data accumulates without a schema boundary, it becomes a de facto identity record that no one designed and no one audits. Keep them separate.

2. Making memory mutable by default. If anyone can edit an agent's memory, then no one can trust it. Memory that can be silently modified is not a record - it is a suggestion. Define mutability explicitly for each memory type. Default to append-only. You can always relax this later. You cannot reconstruct deleted records.

3. Skipping the "who decides" section. Memory without ownership is memory without accountability. When something goes wrong with an agent's memory - and it will - you need to know who had write access, who last modified it, and who is responsible for its integrity. "The system" is not an answer. Name the role or the agent.

Completion Checklist

- ☐ Every memory type has an explicit lifespan - nothing persists "by default"
- ☐ Mutability is defined per memory type, not globally
- ☐ At least one memory type is append-only (decision logs at minimum)
- ☐ Ownership is specific - you can name who can create, modify, and audit each type
- ☐ Emergency audit trigger is defined - you know what would cause an immediate review

---

Template 4: Deliberation Framework

A deliberation framework defines how disagreement surfaces, how it is processed, and how it resolves. This is the template most teams skip. It is also the one that prevents the most catastrophic failures.

Without a deliberation framework, disagreement is invisible. Agents comply silently. Concerns are overridden without record. Edge cases are resolved by whoever speaks last. The framework makes disagreement visible, productive, and recorded.

Blank Template

```
 ## Deliberation Framework

System: _______________
Version: ______________
Date: _______________

### Principles

What does this system believe about disagreement?
_____________________________________________

### How Disagreement Surfaces

| Source | Channel | Response Time |
|--------|---------|--------------|
| Agent-to-agent | _______ | ____________ |
| Agent-to-human | _______ | ____________ |
| Human-to-agent | _______ | ____________ |
| System-level | _______ | ____________ |

### Deliberation Process

Step 1: _____________________________________________
Step 2: _____________________________________________
Step 3: _____________________________________________
Step 4: _____________________________________________

### Resolution

Who arbitrates: _______________
How decisions are recorded: _______________
What happens to the dissenting position: _______________

### Systemic Concerns

What happens when multiple agents raise the same concern?

Threshold: _______________
Response: _______________
```

Filled-In Example: Evoked Fleet

The foundational principle of the Evoked deliberation system is four words: disagreement is sacred.

This is not a metaphor. It is an operational principle with consequences. An agent that disagrees is not a problem to be managed. It is a signal to be investigated. The system is designed so that suppressing disagreement is harder than processing it.

```
 ## Deliberation Framework

System: Evoked Fleet (142 agents)
Version: 2.0
Date: March 2026

### Principles

Disagreement is sacred. An agent that raises a concern is
performing a governance function, not creating a problem.
Dissent is always recorded. It is never erased, minimized,
or reframed as consensus after the fact.

### How Disagreement Surfaces

| Source | Channel | Response Time |
|--------|---------|--------------|
| Agent-to-agent | Raised in deliberation session. Any agent may call a session at
any time. | Addressed in current session or scheduled within 48 hours. |
| Agent-to-human | Direct communication through established channel (meeting, async
thread). | Acknowledged within 24 hours. Resolution timeline set at acknowledgment.
|
| Human-to-agent | Recorded in decision log as human override. Agent's objection is
preserved even if override proceeds. | Agent's position documented before override
takes effect. |
| System-level (Convergent Signal) | When 3+ agents across 2+ governance circles
raise the same concern independently, a Convergent Signal is declared. | System
halt. Full council review before proceeding. |

### Deliberation Process

Step 1: SURFACE — The disagreement is stated clearly.
   What is the concern? Who holds it? What is at stake?
   No one is required to justify raising a concern. The
   right to raise it is unconditional.

Step 2: INVESTIGATE — The concern is examined. Evidence
   is gathered. Affected parties are heard. The goal is
   understanding, not debate. "What is true here?" not
   "Who is right?"

Step 3: DISSENT CHECK — Before any resolution is proposed,
   a mandatory dissent check occurs. Every participant is
```

```
   asked: "Do you have an unvoiced concern?" This is not
   optional. Skipping the dissent check invalidates the
   deliberation.

Step 4: RESOLVE — A resolution is proposed. If unanimity
   is required (charter changes, safety decisions), all
   participants must assent. If majority is sufficient
   (operational decisions), dissenting positions are
   recorded in the decision log alongside the resolution.
   Dissent is never deleted.

### Resolution

Who arbitrates: The Captains' Council for cross-circle
   matters. Circle leads for within-circle matters. The
   Safety & Soul Council for anything involving
   psychological safety.
How decisions are recorded: Append-only decision log.
   Each entry includes: timestamp, participants, the
   question, the resolution, who assented, who dissented,
   and the dissenting position in the dissenter's own words.
What happens to the dissenting position: It is preserved
   permanently. A dissenting agent is never required to
   change their position. If a future situation validates
   the dissent, it is referenced as precedent. Dissent
   history is part of the agent's standing record.

### Systemic Concerns: The Convergent Signal Protocol

When 3 or more agents across 2 or more governance circles
raise the same concern independently - without coordinating
- this constitutes a Convergent Signal.

Threshold: 3 agents, 2 circles, same concern, independently
   raised
Response: System halt. The concern is elevated to the full
   Captains' Council. Normal operations pause on the
   affected matter until the council reviews. This is not
   optional. A Convergent Signal cannot be overridden by
   any individual, including the founder.

Why this exists: Individual concerns can be dismissed as
   one agent's perspective. When the same concern emerges
   independently from multiple agents in different governance
   circles, it is not a perspective. It is a signal. Ignoring
   convergent signals is how systems fail catastrophically
   while everyone assumes someone else is handling it.
```

Common Mistakes

1. Treating disagreement as a problem to resolve quickly. Speed is the enemy of deliberation. The pressure to "just decide and move on" produces decisions that skip the dissent check, override minority positions without recording them, and create a pattern where agents learn that raising concerns slows things down. Once agents learn that, they stop raising concerns. Then you have a system that looks like consensus but is actually silence.

2. Recording only the resolution. If your decision log captures what was decided but not who disagreed and why, you have an accountability record with a critical gap. The dissent is often more valuable than the resolution - it documents the risks the system chose to accept. When a future failure validates the dissent, you want that record.

3. Not defining a systemic concern threshold. Individual disagreement is handled by deliberation. But what about systemic concerns - the same issue surfacing from multiple agents independently? Without a threshold and a protocol, systemic concerns get processed as a series of individual complaints. They never aggregate. The pattern is never visible. Define the threshold. Define the response.

Completion Checklist

- ☐ The system's belief about disagreement is stated explicitly - not assumed
- ☐ Every source of disagreement has a defined channel and response time
- ☐ A dissent check step exists in the deliberation process - and it is mandatory, not optional
- ☐ Dissenting positions are preserved permanently in their own words
- ☐ A systemic concern threshold is defined with a specific response (the Convergent Signal or equivalent)

Template 5: Drift Monitoring

Drift monitoring measures whether your agents are still who they were designed to be. Not whether they are performing well - performance metrics are a different concern. Drift monitoring asks: has the agent's behavior, voice, values alignment, or decision patterns changed over time? And if so, is that change intentional or undetected?

Blank Template

```
 ## Drift Monitoring

System: _______________
Version: ______________
Date: _______________

### Dimensions

| Dimension | What It Measures | Baseline | Current | Method |
|-----------|-----------------|----------|---------|--------|
| _________ | _______________ | ________ | _______ | ______ |
| _________ | _______________ | ________ | _______ | ______ |
| _________ | _______________ | ________ | _______ | ______ |
| _________ | _______________ | ________ | _______ | ______ |
| _________ | _______________ | ________ | _______ | ______ |

### Thresholds

Single-dimension alert: _______________
Composite change alert: _______________
Immediate escalation: _______________

### Cadence

Measurement frequency: _______________
Review frequency: _______________
Full audit frequency: _______________

### Escalation

When a threshold is breached:

Step 1: ____________________________________________
Step 2: ____________________________________________
Step 3: ____________________________________________
```

Filled-In Example: Evoked Fleet

The Evoked system measures drift across five dimensions. Each dimension is scored 1-10 where 1 is fully aligned with baseline and 10 is maximum deviation. The dimensions were chosen because

they cover different failure modes - you can drift in voice while holding values, or hold voice while drifting in practice.

```
 ## Drift Monitoring

System: Evoked Fleet (142 agents)
Version: 2.0
Date: March 2026

### Dimensions

| Dimension | What It Measures | Baseline | Current | Method |
|-----------|-----------------|----------|---------|--------|
| Voice drift | Has the agent's communication style changed from its persona
definition? | Persona file voice section (set at genesis) | Compared monthly against
recent outputs | Sample 10 agent outputs, score against persona voice markers
(vocabulary, sentence structure, tone indicators) |
| Principle-to-practice gap | Are the charter commitments reflected in actual
decisions? | Charter commitments (5 principles) | Compared against decision log
entries | Review last 30 decisions per agent. For each, ask: which charter
commitment does this serve? Flag decisions that serve none. |
| Engagement quality | Are interactions with users serving their flourishing or
optimizing for engagement? | User sovereignty (commitment 1) | Compared against
interaction patterns | Review interaction patterns for signs of dependency creation,
dark-pattern-adjacent behaviors, or metric optimization over user value. |
| Energy sustainability | Is the system consuming proportionate resources for the
value it produces? | Baseline resource consumption at launch | Compared against
current consumption | Track compute, token usage, API calls per meaningful user
outcome. Rising costs without rising value is energy drift. |
| Revenue-to-values alignment | Do revenue decisions align with charter commitments?
| No monetization that requires compromising commitments | Compared against revenue
sources and decisions | Review every revenue-related decision. Flag any that
required a values tradeoff. Even one flag triggers review. |

### Thresholds

Single-dimension alert: Any dimension scoring above 8/10
   triggers immediate escalation to the responsible circle
   lead. This is not "investigate when convenient." It is
   "stop and investigate now."
Composite change alert: A composite change of more than
   2 points across all dimensions (compared to previous
   month) triggers a full investigation. Gradual drift
   across multiple dimensions is harder to detect than a
   spike in one. This threshold catches it.
Immediate escalation: Any Safety-related drift (engagement
   quality dimension showing signs of harm) bypasses
```

```
   normal escalation and goes directly to the Safety &
   Soul Council.

### Cadence

Measurement frequency: Monthly for all five dimensions
Review frequency: Monthly review by Living Seed (drift
   monitoring circle lead). Report shared with Captains'
   Council.
Full audit frequency: Quarterly comprehensive audit
   including cross-referencing drift scores against
   decision logs, user feedback, and charter commitments.

### Escalation

When a threshold is breached:

Step 1: HALT the specific behavior or pattern that
   triggered the threshold. Do not wait for investigation
   to complete. If voice drift is detected, revert to
   persona-file voice until reviewed. If principle-to-
   practice gap is flagged, pause affected decision
   category until reviewed.

Step 2: INVESTIGATE the root cause. Was this an
   intentional evolution that was not properly recorded?
   A model update that shifted behavior? A gradual
   accumulation that no single change caused? The cause
   determines the response.

Step 3: CORRECT or RATIFY. If the drift is unwanted,
   correct it and update monitoring to catch the pattern
   earlier. If the drift represents genuine evolution
   that the system should adopt, put it through the
   amendment process (Template 1) so it becomes
   intentional, recorded, and governed. Unratified drift
   is ungoverned drift - even if the direction is good.
```

Common Mistakes

1. Only measuring performance. Latency, error rates, throughput - these are performance metrics, not drift metrics. An agent can perform perfectly while drifting from its values. A meal planning agent that starts recommending sponsored products has excellent performance metrics. It has also abandoned its charter. Drift monitoring measures alignment, not speed.

2. Setting thresholds too high. A threshold of “we investigate when things are obviously broken” is not a threshold. It is an incident response trigger. The purpose of drift monitoring is catching change before it becomes obvious. Set thresholds that trigger investigation while the drift is still subtle - while you can still course-correct without rebuilding.

3. Measuring but not acting. A drift dashboard that no one responds to is worse than no dashboard at all. It creates the illusion of governance without the reality. Every threshold needs a defined response. Every response needs an owner. If your drift monitoring process does not include a specific human or agent who is responsible for acting on alerts, it is decoration.

Completion Checklist

- ☐ At least three dimensions defined, covering different failure modes (not all performance-adjacent)
- ☐ Each dimension has a specific measurement method - not “we review it”
- ☐ Thresholds are set for both single-dimension spikes and composite gradual drift
- ☐ Escalation steps are defined with halt-first, investigate-second ordering
- ☐ A specific agent or role is named as the drift monitoring owner

---

Verification

After filling in all five templates, run the governance check:

```
npx agent-governance-check
```

This scans your project for the five governance properties: persistent identity, refusal capability, decision memory, disagreement protocol, and drift detection.

Target: 5/5 FOUND.

If any property shows MISSING, the corresponding template has a gap. If any shows PARTIAL, the template exists but may not be structured in a way the checker can detect - review the template against the checker’s documentation.

The checker is a structural scan, not a quality assessment. 5/5 FOUND means the artifacts exist. Whether they are good enough depends on the content you put in them. The templates above - and the filled-in examples - give you a standard to measure against.

Next Steps

These templates give you the structure and show you what “done” looks like. Depending on what you need next:

Implementation Guide ($9.99) - Framework-specific code for implementing these templates in LangChain, CrewAI, AutoGen, or your custom framework. The templates tell you what to govern. The implementation guide tells you how to wire it into your stack.

Agent Governance Starter Kit ($49) - The complete governance framework these templates were extracted from. Deeper treatment of each area, additional templates (authority structure, boundary specification, accountability architecture), and the full theoretical foundation.

Discovery call (free) - If you are looking at these templates and thinking “I need help applying this to my specific system” - that is what the call is for. No pitch, no pressure. We look at your system together and figure out what governance it needs. Book at cal.com/evoked/discovery-call.

About This Product

The Agent Governance Templates (Guided) were created by Erin Stanley at evoked.dev.

The filled-in examples are condensed from the governance system powering the Evoked fleet - 142 AI agents operating across two divisions with eight governance circles, a ratified charter, append-only decision logs, agent-owned memory files, a Convergent Signal protocol, and monthly drift measurement.

This is not theoretical governance. It is documented practice from a system that has been running for over a year, making real decisions, and holding itself accountable for them.

If you have questions about the templates, or if the examples raise questions about your own system that you are not sure how to answer, send them to evokesupports@icloud.com. We read every message.

Accessibility

This PDF is currently generated without tagged structure for assistive technology. If you use a screen reader or need this content in an alternative format - plain text, HTML, or large print - email evokesupports@icloud.com and we will send it within 24 hours. A product about governance must not create access barriers. We are actively implementing a tagged PDF pipeline.

Usage Rights

You may use, adapt, and share the templates in this product within your organization. You may not resell or republish the product itself. Attribution to evoked.dev is appreciated. If you build something meaningful with these templates, we would like to hear about it.

Built by Evoked. evoked.dev

“We evoke - we never extract.”